MAST30027: Modern Applied Statistics

Assignment 3, 2026 — Recovering Lost Reef Classifications: Sample Solution

AUTHOR
Instructor solution

About this solution

This is one defensible solution. The derivations in Problem 1 Parts (a)–(c) and Problem 2 Parts (a)–(b) have essentially one correct answer up to notation, but students may present the algebra differently, may order the mixture components differently, and may write the EM implementation in many valid ways. Award full marks where the working is correct, complete, and justified.

```
X  <- scan(file = "assignment3_prob1_2026.txt", what = double())
X0 <- scan(file = "assignment3_prob2_2026.txt", what = double())

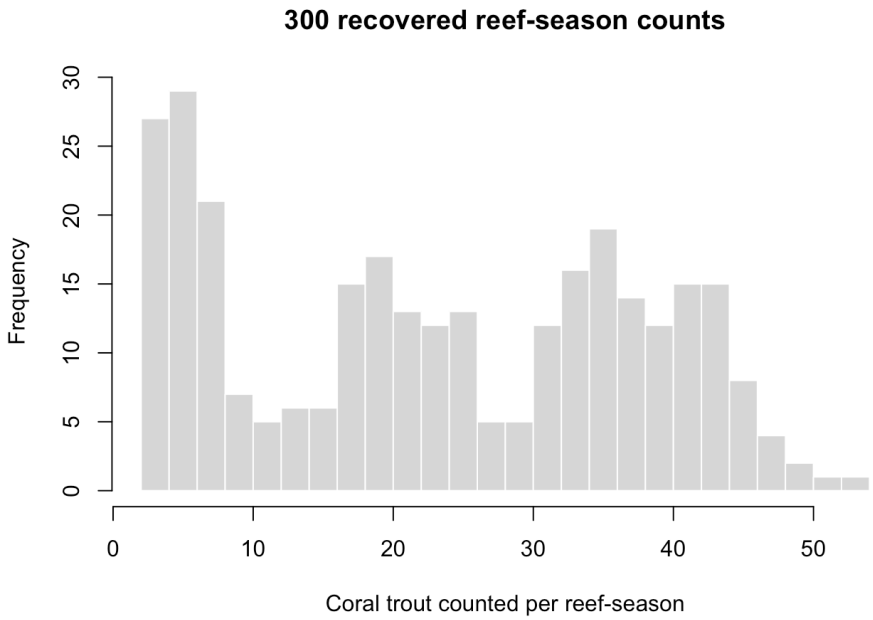
length(X)
```

```
[1] 300
```

```
length(X0)
```

```
[1] 100
```

```
hist(X,
  breaks = 30, col = "grey85", border = "white",
  main = "300 recovered reef-season counts",
  xlab = "Coral trout counted per reef-season"
)
```



Recovered archive counts. The three modes correspond to the three unobserved management classes.

The histogram is clearly multimodal, which is consistent with the client's belief that three distinct management classes are present.

Problem 1: Recovering the three classes

Part (a): Expected complete log-likelihood (5 marks) — MARKED

Let $\theta = (\pi_1, \pi_2, \lambda_1, \lambda_2, \lambda_3)$, write $\pi_3 = 1 - \pi_1 - \pi_2$, and let $n = 300$.

The complete data are (X_i, Z_i) for $i = 1, \ldots, n$. Since the pairs are independent,

$$\begin{aligned} p(X, Z \mid \theta) &= \prod_{i=1}^{n} p(X_i \mid Z_i, \theta)\, p(Z_i \mid \theta) \\ &= \prod_{i=1}^{n} \prod_{k=1}^{3} \left[p(X_i \mid Z_i = k, \theta)\, p(Z_i = k \mid \theta) \right]^{I(Z_i = k)}, \end{aligned}$$

where the indicator exponent selects the single term corresponding to the class that reef i actually belongs to. Taking logarithms and substituting the Poisson probability mass function $p(X_i \mid Z_i = k, \theta) = \lambda_k^{x_i} e^{-\lambda_k} / x_i!$ and $p(Z_i = k \mid \theta) = \pi_k$,

$$\log p(X, Z \mid \theta) = \sum_{i=1}^{n} \sum_{k=1}^{3} I_{(Z_i=k)} \left[\log p(X_i \mid Z_i = k, \theta) + \log p(Z_i = k \mid \theta)\right]$$
$$= \sum_{i=1}^{n} \sum_{k=1}^{3} I_{(Z_i=k)} \left[x_i \log \lambda_k - \lambda_k - \log(x_i!) + \log \pi_k\right].$$

The only random quantity here is $I_{(Z_i=k)}$, and its expectation given the observed data and θ^0 is

$$E_{Z|X,\theta^0}\left[I_{(Z_i=k)}\right] = P(Z_i = k \mid X_i, \theta^0) =: P_{ik},$$

the **responsibility** that component k takes for observation i. Therefore

$$\boxed{Q(\theta, \theta^0) = \sum_{i=1}^{n} \sum_{k=1}^{3} P_{ik} \left[x_i \log \lambda_k - \lambda_k - \log(x_i!) + \log \pi_k\right].}$$

The term $-\log(x_i!)$ does not depend on θ and may be dropped when maximising, but it must be retained if Q itself is to be evaluated.

Part (b): E-step (3 marks)

The E-step computes P_{ik} at the current parameter value $\theta^0 = (\pi_1^0, \pi_2^0, \lambda_1^0, \lambda_2^0, \lambda_3^0)$. By Bayes' theorem, and using the independence of the observations,

$$P_{ik} = P(Z_i = k \mid X_i, \theta^0) = \frac{p(X_i, Z_i = k \mid \theta^0)}{\sum_{j=1}^{3} p(X_i, Z_i = j \mid \theta^0)}$$
$$= \frac{p(X_i \mid Z_i = k, \theta^0)\, p(Z_i = k \mid \theta^0)}{\sum_{j=1}^{3} p(X_i \mid Z_i = j, \theta^0)\, p(Z_i = j \mid \theta^0)}$$
$$= \frac{\pi_k^0 \, (\lambda_k^0)^{x_i} e^{-\lambda_k^0}}{\sum_{j=1}^{3} \pi_j^0 \, (\lambda_j^0)^{x_i} e^{-\lambda_j^0}},$$

where the factor $1/x_i!$ is common to numerator and denominator and cancels.

Interpretation for the client. For a single reef with observed count x_i, P_{ik} is the probability that this reef belonged to management class k, given its count and the current estimates of how common each class is and how abundant coral trout typically are in each. It is exactly the quantity the Monitoring Manager asked for when they asked how confident we can be about an individual reef.

Part (c): M-step (5 marks)

The M-step maximises $Q(\theta, \theta^0)$ over θ, treating the P_{ik} as fixed.

Mixture proportions. These are constrained by $\pi_1 + \pi_2 + \pi_3 = 1$, so we use a Lagrange multiplier:

$$L = \sum_{i=1}^{n} \sum_{k=1}^{3} P_{ik} \left[x_i \log \lambda_k - \lambda_k - \log(x_i!) + \log \pi_k\right] + \nu \left(\pi_1 + \pi_2 + \pi_3 - 1\right).$$

Differentiating with respect to π_k and setting to zero,

$$\frac{\partial L}{\partial \pi_k} = \sum_{i=1}^{n} \frac{P_{ik}}{\pi_k} + \nu = 0 \qquad \Longrightarrow \qquad \pi_k = -\frac{1}{\nu} \sum_{i=1}^{n} P_{ik}.$$

Summing over k and using $\sum_k \pi_k = 1$ together with $\sum_{k=1}^{3} P_{ik} = 1$ for every i,

$$1 = -\frac{1}{\nu} \sum_{i=1}^{n} \sum_{k=1}^{3} P_{ik} = -\frac{1}{\nu} \sum_{i=1}^{n} 1 = -\frac{n}{\nu},$$

so $\nu = -n$ and

$$\boxed{\hat{\pi}_k = \frac{1}{n} \sum_{i=1}^{n} P_{ik}, \qquad k = 1, 2, 3.}$$

Component means. These are unconstrained, and each λ_k appears in only one term of the sum over k:

$$\frac{\partial Q}{\partial \lambda_k} = \sum_{i=1}^{n} P_{ik} \left(\frac{x_i}{\lambda_k} - 1 \right) = 0 \qquad \Longrightarrow \qquad \boxed{\hat{\lambda}_k = \frac{\sum_{i=1}^{n} P_{ik}\, x_i}{\sum_{i=1}^{n} P_{ik}}.}$$

This is a maximum, since

$$\frac{\partial^2 Q}{\partial \lambda_k^2} = -\frac{1}{\lambda_k^2} \sum_{i=1}^{n} P_{ik}\, x_i < 0.$$

Note that using a Lagrange multiplier is one way of solving the problem, but the result can also be obtained by directly differentiating with respect to π_1 and π_2.

Comment on the form of $\hat{\lambda}_k$. It is a *weighted* sample mean of all n observations, with weights given by the responsibilities. If the class labels had been observed, the MLE of λ_k would be the ordinary sample mean of the observations in class k — which is the special case where every P_{ik} is 0 or 1. The EM algorithm replaces hard group membership with fractional membership: every reef contributes to every component's mean, in proportion to how likely it is to have come from that component. Similarly, $\hat{\pi}_k$ is the average responsibility, which replaces the observed class proportion.

Part (d): Implementation (5 marks) — MARKED

```
# P(X_i, Z_i = k) for each observation and component, as an n x
compute.prob.x.z <- function(X, pi.curr, lambda.curr) {
  sapply(1:3, function(k) dpois(X, lambda.curr[k]) * pi.curr[k]
}

# Incomplete (observed-data) log-likelihood for the Problem 1 m
#   sum_i log( sum_k pi_k * f(x_i; lambda_k) )
compute.log.lik <- function(X, pi.curr, lambda.curr) {
  sum(log(rowSums(compute.prob.x.z(X, pi.curr, lambda.curr))))
}
```

```r
# EM algorithm for a three-component Poisson mixture
em.mix.pois <- function(X, pi.init, lambda.init,
                        max.iter = 100, eps = 0.00001) {
  pi.curr     <- pi.init
  lambda.curr <- lambda.init
  log.lik     <- compute.log.lik(X, pi.curr, lambda.curr)
  log.lik.vec <- log.lik
  n           <- length(X)

  for (i in 1:max.iter) {
    # E-step: responsibilities P_ik
    prob.x.z <- compute.prob.x.z(X, pi.curr, lambda.curr)
    P.ik     <- prob.x.z / rowSums(prob.x.z)

    # M-step
    pi.curr     <- colSums(P.ik) / n
    lambda.curr <- colSums(P.ik * X) / colSums(P.ik)

    # Convergence check on the incomplete log-likelihood
    log.lik.new <- compute.log.lik(X, pi.curr, lambda.curr)
    log.lik.vec <- c(log.lik.vec, log.lik.new)
    if (abs(log.lik.new - log.lik) < eps) break
    log.lik <- log.lik.new
  }

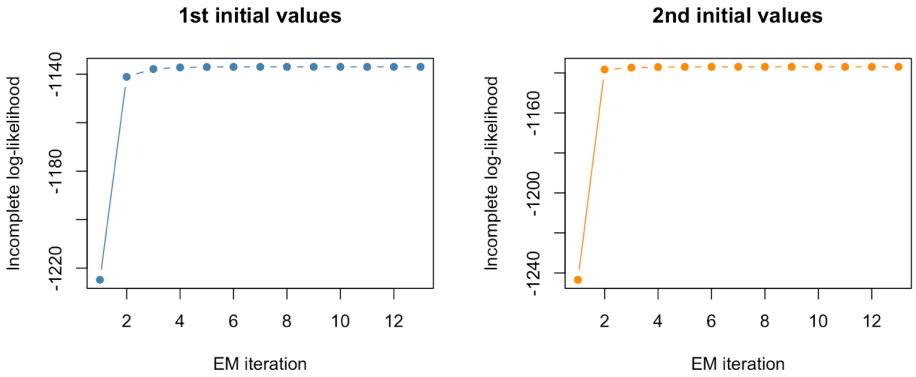
  list(
    pi = pi.curr, lambda = lambda.curr,
    log.lik = log.lik.new, log.lik.vec = log.lik.vec,
    P.ik = P.ik, iter = i
  )
}
```

```r
fit1a <- em.mix.pois(X, pi.init = c(0.3, 0.3, 0.4), lambda.init
fit1b <- em.mix.pois(X, pi.init = c(0.1, 0.2, 0.7), lambda.init

rbind(
  "1st initial values" = c(pi = round(fit1a$pi, 4), lambda = rou
                           loglik = round(fit1a$log.lik, 4), ite
  "2nd initial values" = c(pi = round(fit1b$pi, 4), lambda = rou
                           loglik = round(fit1b$log.lik, 4), ite
)
```

```
                     pi1    pi2    pi3 lambda1 lambda2 lambda3
loglik iter
1st initial values 0.2906 0.2851 0.4242  5.9055 20.4273 37.9804
-1136.962   12
2nd initial values 0.2907 0.2852 0.4242  5.9059 20.4300 37.9822
-1136.962   12
```

```r
par(mfrow = c(1, 2))
plot(fit1a$log.lik.vec, type = "b", pch = 16, col = "steelblue",
     xlab = "EM iteration", ylab = "Incomplete log-likelihood",
     main = "1st initial values")
plot(fit1b$log.lik.vec, type = "b", pch = 16, col = "darkorange"
     xlab = "EM iteration", ylab = "Incomplete log-likelihood",
     main = "2nd initial values")
```



Incomplete log-likelihood against EM iteration, for both sets of initial values. Both runs increase at every step and converge to the same value.

```
# The log-likelihood must increase at every EM step
all(diff(fit1a$log.lik.vec) > 0)
```

```
[1] TRUE
```

```
all(diff(fit1b$log.lik.vec) > 0)
```

```
[1] TRUE
```

```
best1 <- if (fit1a$log.lik >= fit1b$log.lik) fit1a else fit1b
```

Both runs converge to the same incomplete log-likelihood of -1136.962, taking 12 and 12 iterations respectively, well inside the limit of 100. The two starting points therefore agree, and we report:

```
data.frame(
  component = 1:3,
  pi.hat     = round(best1$pi, 4),
  lambda.hat = round(best1$lambda, 3)
)
```

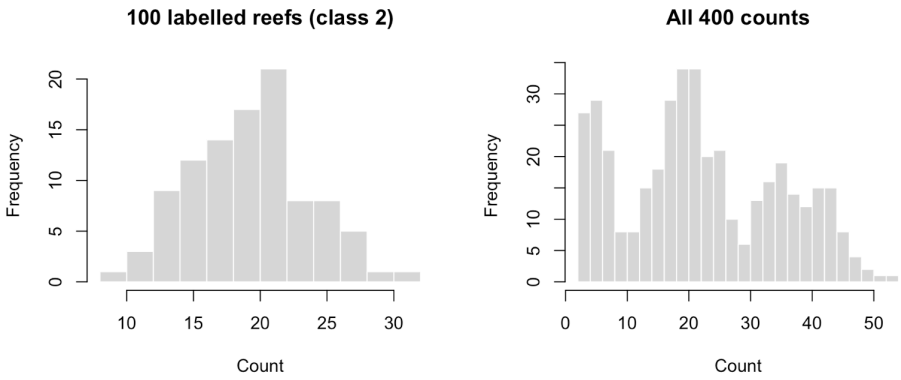
```
  component pi.hat lambda.hat
1         1 0.2907      5.906
2         2 0.2852     20.430
3         3 0.4242     37.982
```

So $\hat{\pi} \approx (0.291, 0.285, 0.424)$ and $\hat{\lambda} \approx (5.91, 20.43, 37.98)$.

What we would conclude if the log-likelihood did not increase. The EM algorithm guarantees that the incomplete log-likelihood is non-decreasing at every iteration. A decrease is therefore not a property of the data or of a poor starting value — it is proof of an error in the implementation, most often a mis-derived M-step update, an E-step and M-step applied in the wrong order, or the log-likelihood being evaluated at the wrong parameter values. It would mean the code must be debugged, not that the model should be changed.

Problem 2: Using the recovered labelled surveys

```r
par(mfrow = c(1, 2))
hist(X0, breaks = 15, col = "grey85", border = "white",
     main = "100 labelled reefs (class 2)", xlab = "Count")
hist(c(X, X0), breaks = 30, col = "grey85", border = "white",
     main = "All 400 counts", xlab = "Count")
```



The recovered labelled batch (left) and the combined data (right).

The labelled batch is unimodal and sits squarely under the middle mode of the archive data, as it should if those reefs all belong to class 2. Its sample mean is 19.59.

Part (a): Expected complete log-likelihood (5 marks)

Write $n = 300$ and $n' = 100$. The class label is unobserved only for the first batch; for $i = 301, \dots, 400$ we are *told* the reef belongs to class 2, so there is no latent variable to introduce for those observations. This is why $Z = (Z_1, \dots, Z_{300})$ has 300 components while $X = (X_1, \dots, X_{400})$ has 400.

By independence, the complete-data likelihood factorises into the mixture part and the labelled part:

$$p(X, Z \mid \theta) = \prod_{i=1}^{n} \prod_{k=1}^{3} \left[p(X_i \mid Z_i = k, \theta)\, \pi_k \right]^{I(Z_i = k)} \times \prod_{i=n+1}^{n+n'} p(X_i \mid \theta),$$

where $p(X_i \mid \theta) = \lambda_2^{x_i} e^{-\lambda_2} / x_i!$ for $i > n$. Taking logarithms,

$$\log p(X, Z \mid \theta) = \sum_{i=1}^{n} \sum_{k=1}^{3} I_{(Z_i = k)} \left[x_i \log \lambda_k - \lambda_k - \log(x_i!) + \log \pi_k \right] + \sum_{i=n+1}^{n+n'} \left[x_i \log \lambda_2 - \lambda_2 - \log(x_i!) \right].$$

The second sum contains no Z_i, so it is a constant with respect to the expectation over $Z \mid X, \theta^0$ and passes through unchanged. Taking the expectation of the first

sum exactly as in Problem 1 Part (a),

$$\boxed{\begin{aligned} Q(\theta, \theta^0) = &\sum_{i=1}^{n} \sum_{k=1}^{3} P_{ik} \left[x_i \log \lambda_k - \lambda_k - \log(x_i!) + \log \pi_k \right] \\ &+ \sum_{i=n+1}^{n+n'} \left[x_i \log \lambda_2 - \lambda_2 - \log(x_i!) \right], \end{aligned}}$$

with $P_{ik} = P(Z_i = k \mid X_i, \theta^0)$ defined for $i = 1, \ldots, n$ only.

The labelled batch therefore enters Q through a term that involves λ_2 and nothing else. This observation drives the whole of part (b).

Part (b): E-step and M-step (5 marks) — MARKED

E-step. The responsibilities are needed only for the 300 unlabelled reefs, and their expression is exactly as in Problem 1 Part (b):

$$P_{ik} = \frac{\pi_k^0 \, (\lambda_k^0)^{x_i} e^{-\lambda_k^0}}{\sum_{j=1}^{3} \pi_j^0 \, (\lambda_j^0)^{x_i} e^{-\lambda_j^0}}, \qquad i = 1, \ldots, 300.$$

For $i = 301, \ldots, 400$ the class is known, so the "responsibility" is degenerate: it is 1 for component 2 and 0 otherwise. Nothing needs to be computed.

M-step. Comparing the two expressions for Q, the extra term involves only λ_2. Hence:

- The update for π_k is **unchanged**:

$$\hat{\pi}_k = \frac{1}{n} \sum_{i=1}^{n} P_{ik}, \qquad n = 300.$$

  The divisor is 300, not 400. The labelled reefs were not drawn from the mixture — they were selected *because* they belong to class 2 — so they carry no information about how common each class is in the archive. Including them would bias $\hat{\pi}_2$ upwards.

- The updates for λ_1 and λ_3 are **unchanged**:

$$\hat{\lambda}_k = \frac{\sum_{i=1}^{n} P_{ik} \, x_i}{\sum_{i=1}^{n} P_{ik}}, \qquad k = 1, 3.$$

- The update for λ_2 **changes**. Differentiating,

$$\frac{\partial Q}{\partial \lambda_2} = \sum_{i=1}^{n} P_{i2} \left(\frac{x_i}{\lambda_2} - 1 \right) + \sum_{i=n+1}^{n+n'} \left(\frac{x_i}{\lambda_2} - 1 \right) = 0,$$

  giving

$$\boxed{\hat{\lambda}_2 = \frac{\sum_{i=1}^{n} P_{i2} \, x_i + \sum_{i=n+1}^{n+n'} x_i}{\sum_{i=1}^{n} P_{i2} + n'}.}$$

What the λ_2 update is averaging over. It is still a weighted mean of counts, but now over two kinds of observation. Each unlabelled reef contributes its count with weight P_{i2} — its fractional membership of class 2 — while each labelled reef contributes with weight exactly 1, because we know with certainty that it belongs to class 2. The denominator is the total weight: the *effective* number of class-2 observations among the unlabelled reefs, plus the 100 reefs known to be class 2.

Part (c): Implementation (5 marks) — MARKED

The incomplete log-likelihood is different here

The observed data are $X_1, \ldots, X_{400}$, but only the first 300 are mixture draws. The correct incomplete log-likelihood is

$$\ell(\theta) = \sum_{i=1}^{300} \log \left(\sum_{k=1}^{3} \pi_k \, f(x_i; \lambda_k) \right) + \sum_{i=301}^{400} \log f(x_i; \lambda_2),$$

where $f(\cdot; \lambda)$ is the Poisson probability mass function. Applying the mixture density to all 400 observations is wrong: it would be the log-likelihood of a different model, and the resulting monotonicity check would not be checking the algorithm that is actually being run.

```
# Incomplete log-likelihood for the combined model
compute.log.lik.2 <- function(X, X0, pi.curr, lambda.curr) {
  mixture.part  <- sum(log(rowSums(compute.prob.x.z(X, pi.curr,
  labelled.part <- sum(dpois(X0, lambda.curr[2], log = TRUE))
  mixture.part + labelled.part
}

# EM for the mixture plus a labelled batch known to come from c
em.mix.pois.labelled <- function(X, X0, pi.init, lambda.init,
                                 max.iter = 100, eps = 0.00001)
  pi.curr     <- pi.init
  lambda.curr <- lambda.init
  log.lik     <- compute.log.lik.2(X, X0, pi.curr, lambda.curr)
  log.lik.vec <- log.lik
  n  <- length(X)
  n0 <- length(X0)

  for (i in 1:max.iter) {
    # E-step: responsibilities for the 300 unlabelled reefs only
    prob.x.z <- compute.prob.x.z(X, pi.curr, lambda.curr)
    P.ik     <- prob.x.z / rowSums(prob.x.z)

    # M-step
    pi.curr <- colSums(P.ik) / n                        # diviso
    lambda.curr[1] <- sum(P.ik[, 1] * X) / sum(P.ik[, 1])
    lambda.curr[3] <- sum(P.ik[, 3] * X) / sum(P.ik[, 3])
    lambda.curr[2] <- (sum(P.ik[, 2] * X) + sum(X0)) /
                      (sum(P.ik[, 2]) + n0)             # labell

    log.lik.new <- compute.log.lik.2(X, X0, pi.curr, lambda.cur
    log.lik.vec <- c(log.lik.vec, log.lik.new)
```

```
    if (abs(log.lik.new - log.lik) < eps) break
    log.lik <- log.lik.new
  }

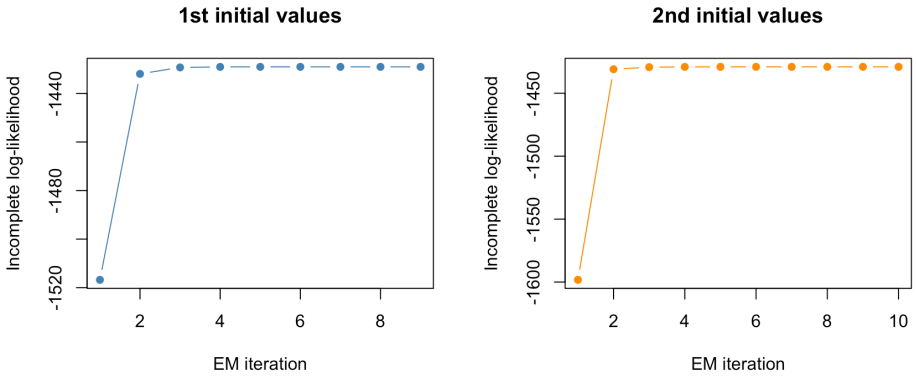
  list(
    pi = pi.curr, lambda = lambda.curr,
    log.lik = log.lik.new, log.lik.vec = log.lik.vec,
    P.ik = P.ik, iter = i
  )
}
```

```
fit2a <- em.mix.pois.labelled(X, X0, c(0.3, 0.3, 0.4), c(3, 20,
fit2b <- em.mix.pois.labelled(X, X0, c(0.1, 0.2, 0.7), c(5, 25,

rbind(
  "1st initial values" = c(pi = round(fit2a$pi, 4), lambda = rou
                           loglik = round(fit2a$log.lik, 4), ite
  "2nd initial values" = c(pi = round(fit2b$pi, 4), lambda = rou
                           loglik = round(fit2b$log.lik, 4), ite
)
```

```
                     pi1    pi2    pi3 lambda1 lambda2 lambda3
loglik iter
1st initial values 0.2885 0.2812 0.4302  5.8675 19.8645 37.8450
-1429.048    8
2nd initial values 0.2885 0.2813 0.4302  5.8676 19.8651 37.8463
-1429.048    9
```

```
par(mfrow = c(1, 2))
plot(fit2a$log.lik.vec, type = "b", pch = 16, col = "steelblue",
     xlab = "EM iteration", ylab = "Incomplete log-likelihood",
     main = "1st initial values")
plot(fit2b$log.lik.vec, type = "b", pch = 16, col = "darkorange"
     xlab = "EM iteration", ylab = "Incomplete log-likelihood",
     main = "2nd initial values")
```



Incomplete log-likelihood against EM iteration for the combined model, for both sets of initial values.

```
all(diff(fit2a$log.lik.vec) > 0)
```

```
[1] TRUE
```

```r
all(diff(fit2b$log.lik.vec) > 0)
```

```
[1] TRUE
```

```r
best2 <- if (fit2a$log.lik >= fit2b$log.lik) fit2a else fit2b

data.frame(
  component  = 1:3,
  pi.hat     = round(best2$pi, 4),
  lambda.hat = round(best2$lambda, 3)
)
```

```
  component pi.hat lambda.hat
1         1 0.2885      5.868
2         2 0.2813     19.865
3         3 0.4302     37.846
```

Both starting points again converge to the same solution, with incomplete log-likelihood -1429.048, in 8 and 9 iterations. The estimates are $\hat{\pi} \approx (0.289, 0.281, 0.43)$ and $\hat{\lambda} \approx (5.87, 19.87, 37.85)$.

Note that the log-likelihood values from Problems 1 and 2 are **not** comparable: they are computed from different data (300 observations versus 400) under different models.

Appendix: robustness check (not required of students)

The two prescribed starting values agree, but it is worth confirming that the likelihood surface has no competing local maximum that a student might land on.

```r
set.seed(30027)
ll.p1 <- replicate(50, {
  st <- sort(runif(3, 1, 50))
  em.mix.pois(X, rep(1/3, 3), st)$log.lik
})
ll.p2 <- replicate(50, {
  st <- sort(runif(3, 1, 50))
  em.mix.pois.labelled(X, X0, rep(1/3, 3), st)$log.lik
})

table(round(ll.p1, 3))
```

```
-1136.962
       50
```

```r
table(round(ll.p2, 3))
```

```
-1429.048
```

```
50
```

All 50 random restarts of each algorithm reach the same maximum, so both problems are well conditioned. A student whose answer differs materially from the values above has an error in their implementation rather than an unlucky starting value.